

[Open source](#) ⓘ

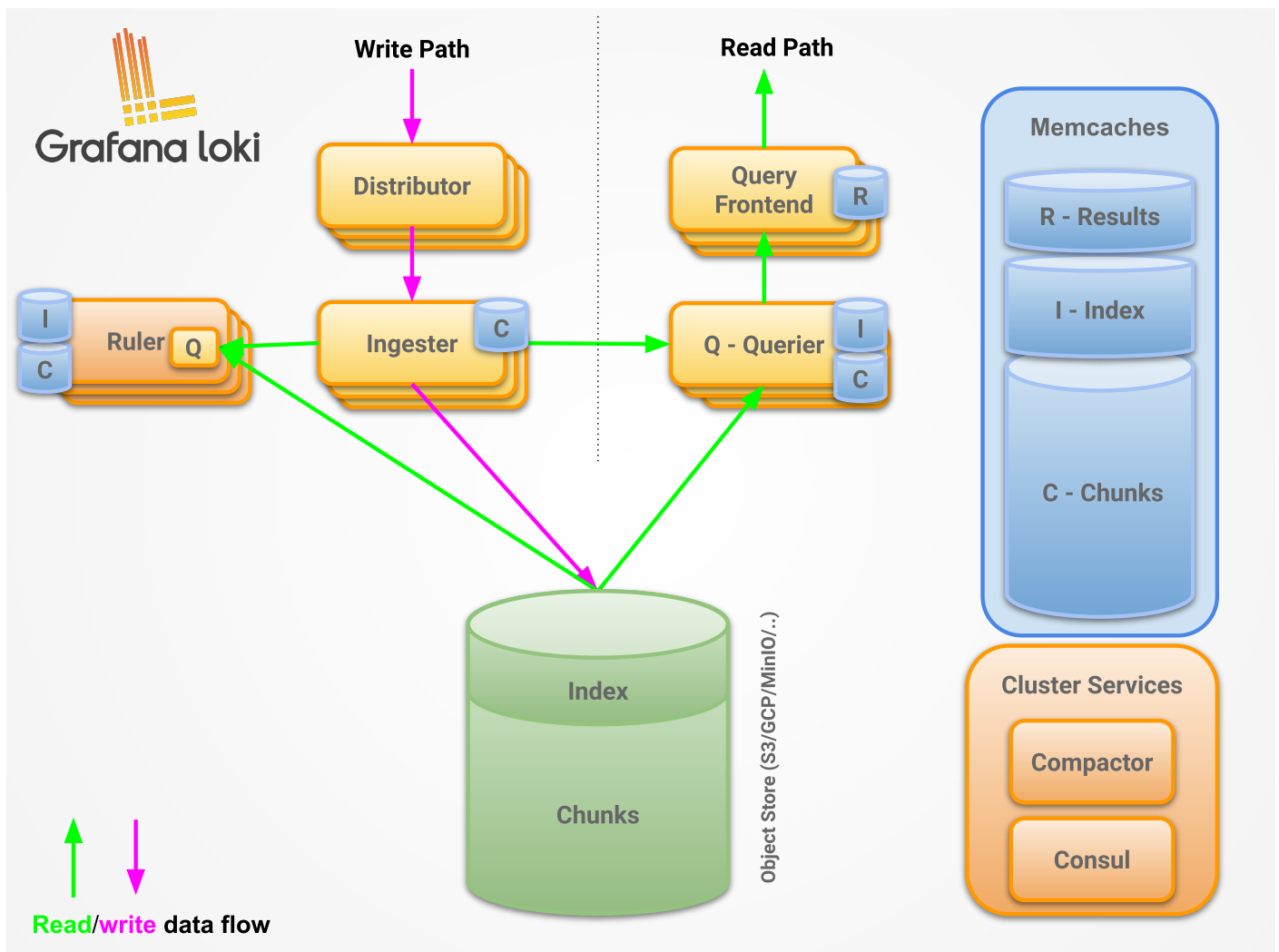
Loki architecture

Grafana Loki has a microservices-based architecture and is designed to run as a horizontally scalable, distributed system. The system has multiple components that can run separately and in parallel. The Grafana Loki design compiles the code for all components into a single binary or Docker image. The `-target` command-line flag controls which component(s) that binary will behave as.

To get started easily, run Grafana Loki in “single binary” mode with all components running simultaneously in one process, or in “simple scalable deployment” mode, which groups components into read, write, and backend parts.

Grafana Loki is designed to easily redeploy a cluster under a different mode as your needs change, with no configuration changes or minimal configuration changes.

For more information, refer to [Deployment modes](#) and [Components](#).



Storage

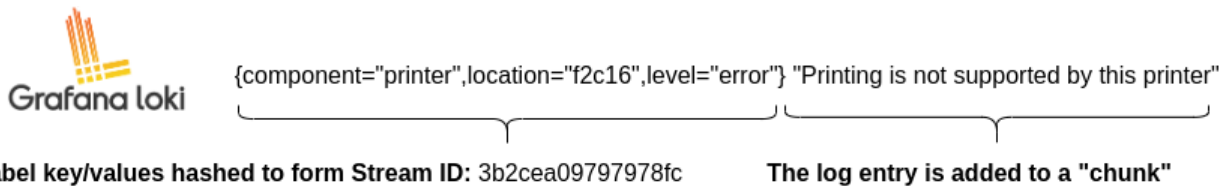
Loki stores all data in a single object storage backend, such as Amazon Simple Storage Service (S3), Google Cloud Storage (GCS), Azure Blob Storage, among others. This mode uses an adapter called **index shipper** (or short **shipper**) to store index (TSDB or BoltDB) files the same way we store chunk files in object storage. This mode of operation became generally available with Loki 2.0 and is fast, cost-effective, and simple. It is where all current and future development lies.

Prior to 2.0, Loki had different storage backends for indexes and chunks. For more information, refer to [Legacy storage](#).

Data format

Grafana Loki has two main file types: **index** and **chunks**.

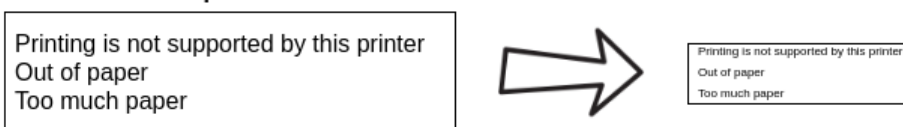
- The **index** is a table of contents of where to find logs for a specific set of labels.
- The **chunk** is a container for log entries for a specific set of labels.



Additional log messages with the same labels are added to the same "chunk":

```
{component="printer",location="f2c16",level="error"} "Out of paper"
{component="printer",location="f2c16",level="error"} "Too much paper"
```

Chunks are filled then compressed and stored:



A separate and small index is kept to lookup chunks

Different label keys or values will hash to a different stream and different chunk:

```
{component="printer",location="f2c16",level="info"} "Consider the environment before printing this log message"
```

fd9a709ddf43a93a

The diagram above shows the high-level overview of the data that is stored in the chunk and data that is stored in the index.

Index format

There are two index formats that are currently supported as single store with index shipper:

- **TSDB** (recommended)

Time Series Database (or short TSDB) is an **index format** originally developed by the maintainers of **Prometheus** for time series (metric) data.

It is extensible and has many advantages over the deprecated BoltDB index. New storage features in Loki are solely available when using TSDB.

- **BoltDB** (deprecated)

Bolt is a low-level, transactional key-value store written in Go.

Chunk format

A chunk is a container for log lines of a stream (unique set of labels) of a specific time range.

The following ASCII diagram describes the chunk format in detail.

Copy

Expand code

	MagicNumber(4b)		version(1b)		encoding (1b)

	#structuredMetadata (uvarint)				

	len(label-1) (uvarint)		label-1 (bytes)		

	len(label-2) (uvarint)		label-2 (bytes)		

	len(label-n) (uvarint)		label-n (bytes)		

	checksum(from #structuredMetadata)				

	block-1 bytes		checksum (4b)		

	block-2 bytes		checksum (4b)		

	block-n bytes		checksum (4b)		

mint and maxt describe the minimum and maximum Unix nanosecond timestamp, respectively.

The structuredMetadata section stores non-repeated strings. It is used to store label names and label values from structured metadata. Note that the labels strings and lengths within the structuredMetadata section are stored compressed.

Block format

A block is comprised of a series of entries, each of which is an individual log line. Note that the bytes of a block are stored compressed. The following is their form when uncompressed:

Copy

	ts (varint)		len (uvarint)		log-1 bytes		len(from #symbols)		#symbols (uvarint)	

	ts (varint)		len (uvarint)		log-2 bytes		len(from #symbols)		#symbols (uvarint)	

	ts (varint)		len (uvarint)		log-3 bytes		len(from #symbols)		#symbols (uvarint)	

	ts (varint)		len (uvarint)		log-n bytes		len(from #symbols)		#symbols (uvarint)	

ts is the Unix nanosecond timestamp of the logs, while len is the length in bytes of the log entry.

Symbols store references to the actual strings containing label names and values in the structuredMetadata section of the chunk.

Write path

On a high level, the write path in Loki works as follows:

- 1 The distributor receives an HTTP POST request with streams and log lines.
- 2 The distributor hashes each stream contained in the request so it can determine the ingester instance to which it needs to be sent based on the information from the consistent hash ring.
- 3 The distributor sends each stream to the appropriate ingester and its replicas (based on the configured replication factor).
- 4 The ingester receives the stream with log lines and creates a chunk or appends to an existing chunk for the stream's data. A chunk is unique per tenant and per label set.

- 5 The ingester acknowledges the write.
- 6 The distributor waits for a majority (quorum) of the ingesters to acknowledge their writes.
- 7 The distributor responds with a success (2xx status code) in case it received at least a quorum of acknowledged writes. or with an error (4xx or 5xx status code) in case write operations failed.

Refer to [Components](#) for a more detailed description of the components involved in the write path.

Read path

On a high level, the read path in Loki works as follows:

- 1 The query frontend receives an HTTP GET request with a LogQL query.
- 2 The query frontend splits the query into sub-queries and passes them to the query scheduler.
- 3 The querier pulls sub-queries from the scheduler.
- 4 The querier passes the query to all ingesters for in-memory data.
- 5 The ingesters return in-memory data matching the query, if any.
- 6 The querier lazily loads data from the backing store and runs the query against it if ingesters returned no or insufficient data.
- 7 The querier iterates over all received data and deduplicates, returning the result of the sub-query to the query frontend.
- 8 The query frontend waits for all sub-queries of a query to be finished and returned by the queriers.
- 9 The query frontend merges the individual results into a final result and return it to the client.

Refer to [Components](#) for a more detailed description of the components involved in the read path.

Multi-tenancy

All data, both in memory and in long-term storage, may be partitioned by a tenant ID, pulled from the `X-Scope-OrgID` HTTP header in the request when Grafana Loki is running in multi-tenant mode. When Loki is **not** in multi-tenant mode, the header is ignored and the tenant ID is set to `fake`, which will appear in the index and in stored chunks.